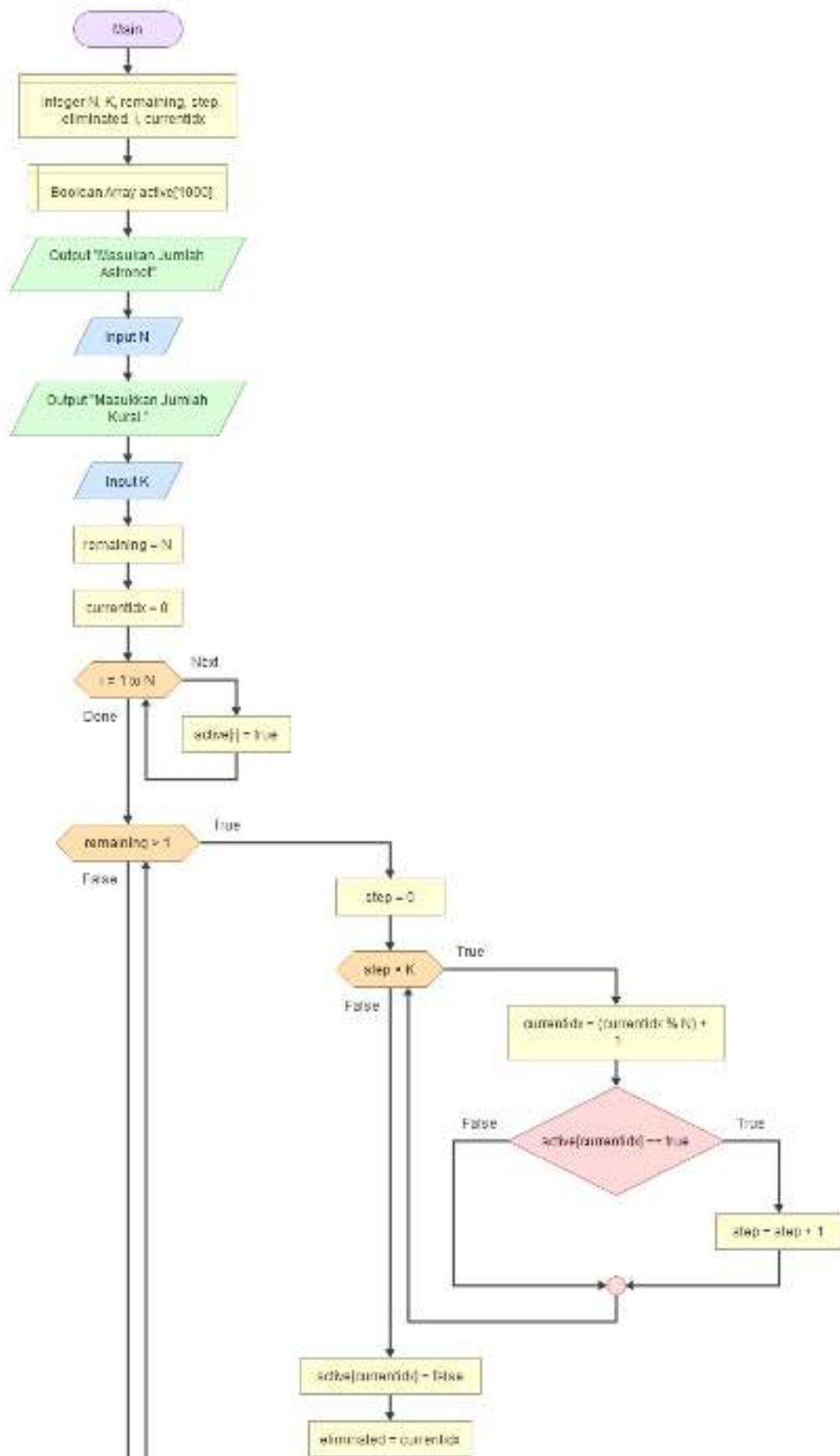
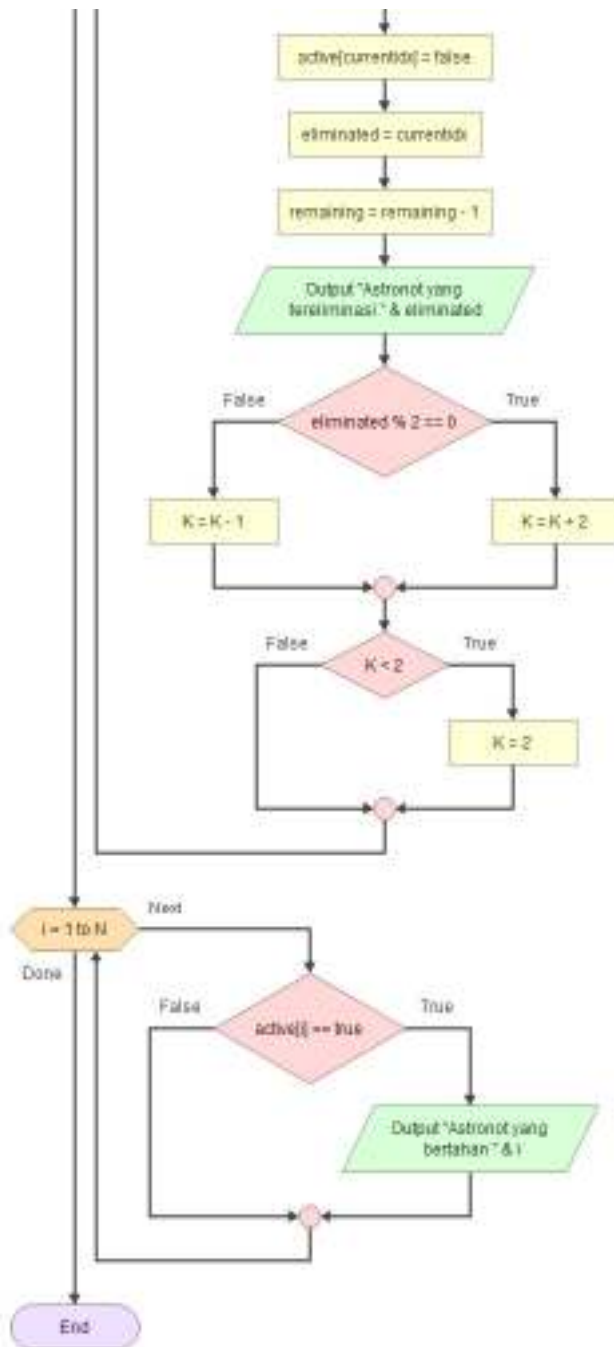


FASE 1

Nama: **Rheindhia Bramantya Cipta** | NIM: **26/575920/TK/65498**

Kelompok: **Amber** | Pemrograman Dasar | TLS 2026





(Flowchart menggunakan aplikasi *Flowgorithm*)

Penjelasan Flowchart Problem 1: The Last Astronaut

1. Gambaran Umum

Flowchart ini menggambarkan permainan eliminasi astronot yang duduk melingkar. Sejumlah astronot dihitung berkeliling, lalu astronot yang jatuh pada hitungan tertentu dieliminasi. Proses berulang sampai tersisa satu astronot.

Ini adalah variasi dari Masalah Josephus. Bedanya, panjang hitungan (K) tidak tetap, melainkan berubah setelah setiap eliminasi tergantung nomor astronot yang tereliminasi.

2. Penjelasan Bagian per Bagian

A. Awal program (Main dan deklarasi)

- Program dimulai dari terminal Main.
- Dideklarasikan tujuh variabel bilangan bulat:
 - N: jumlah astronot.
 - K: panjang hitungan (diinput sebagai "jumlah kursi").
 - remaining: jumlah astronot yang masih tersisa.
 - step: penghitung langkah pada satu putaran hitungan.
 - eliminated: nomor astronot yang baru dieliminasi.
 - i: penghitung untuk perulangan.
 - currentidx: posisi (indeks) astronot saat ini.
- Dideklarasikan juga array boolean bernama active berkapasitas 1000. Array ini menandai apakah astronot nomor tertentu masih aktif (true) atau sudah tereliminasi (false).

B. Input dari pengguna

- Program menampilkan permintaan memasukkan jumlah astronot, lalu membaca nilainya ke N.
- Program menampilkan permintaan memasukkan jumlah kursi, lalu membaca nilainya ke K.

C. Inisialisasi

- remaining diisi sama dengan N, karena awalnya semua astronot masih ada.
- currentidx diisi 0, artinya hitungan dimulai sebelum astronot nomor 1.
- Sebuah perulangan dengan i dari 1 sampai N mengisi `active[i] = true` untuk setiap astronot, sehingga semuanya berstatus aktif.

D. Perulangan utama: selama `remaining > 1`

Perulangan ini terus berjalan selama astronot yang tersisa lebih dari satu. Setiap putaran menghasilkan satu eliminasi.

Langkah 1: menghitung sampai K astronot aktif

- step direset ke 0.
- Perulangan dalam berjalan selama `step < K`. Di dalamnya:

1. `currentidx` maju satu posisi dengan rumus $(\text{currentidx} \bmod N) + 1$. Rumus ini membuat posisi melingkar: setelah astronot nomor N , hitungan kembali ke nomor 1.
2. Program memeriksa apakah astronot di posisi itu masih aktif.
 - Jika aktif (`true`), `step` bertambah 1 karena astronot itu dihitung.
 - Jika tidak aktif (`false`), astronot dilewati dan `step` tidak bertambah.
3. Kembali ke pemeriksaan `step < K`.

Jadi K menghitung hanya astronot yang masih aktif, sedangkan yang sudah tereliminasi diloncati.

Langkah 2: mengeliminasi astronot

Setelah `step` mencapai K , posisi `currentidx` adalah astronot yang terkena eliminasi:

- `active[currentidx]` diubah menjadi `false`, sehingga astronot itu tidak aktif lagi.
- Nomornya disimpan di `eliminated`.
- `remaining` dikurangi 1.
- Program menampilkan pesan "Astronot yang tereliminasi" beserta nomornya.

Langkah 3: menyesuaikan nilai K untuk putaran berikutnya

- Jika `eliminated` genap (sisanya 2 sama dengan 0), maka $K = K + 2$.
- Jika `eliminated` ganjil, maka $K = K - 1$.
- Kemudian ada pengaman: jika $K < 2$, maka K dipaksa menjadi 2. Dengan begitu K tidak pernah di bawah 2 setelah penyesuaian.

Setelah itu alur kembali ke pemeriksaan `remaining > 1`.

E. Menampilkan pemenang

Ketika `remaining` tidak lagi lebih dari 1 (jalur `False`), program masuk ke perulangan `i` dari 1 sampai N :

- Untuk setiap `i`, jika `active[i]` bernilai `true`, program menampilkan "Astronot yang bertahan" beserta nomor `i`.
- Karena hanya satu astronot yang masih aktif, hanya satu nomor yang ditampilkan.

Terakhir program menuju terminal `End`.

3. Contoh Simulasi

Misalkan $N = 5$ dan $K = 3$. Astronot bernomor 1 sampai 5, hitungan dimulai dari posisi 0.

Putaran	K saat itu	Proses hitung	Tereliminasi	Sifat K berikutnya
1	3	Hitung 1, 2, 3	3	Ganjil $3 - 1 = 2$
2	2	Mulai setelah 3: hitung 4, 5	5	Ganjil $2 - 1 = 1$, lalu dipaksa 2
3	2	Setelah 5 kembali ke 1: hitung 1, 2	2	Genap $2 + 2 = 4$
4	4	Dari 2: 3 dilewati, hitung 4, 1, lalu 4, 1 (astronot tereliminasi dilewati)	1	Ganjil $4 - 1 = 3$

Hasil akhir: astronot yang tereliminasi berurutan adalah 3, 5, 2, 1, dan yang bertahan adalah astronot nomor 4.

```

MULAI

    DEKLARASI pesan SEBAGAI String
    DEKLARASI hasil SEBAGAI String = ""

TAMPILKAN "Masukkan pesan rahasia: "

    BACA pesan

UNTUK i DARI 0 SAMPAI panjang(pesan) - 1 LAKUKAN
    JIKA i == 0 MAKA
        TAMBAHKAN pesan[i] KE DALAM hasil
    SEBALIKNYA
        nilaiSekarang = (Nilai ASCII pesan[i]) - (Nilai ASCII
        'A') + 1
        nilaiSebelumnya = (Nilai ASCII pesan[i-1]) - (Nilai
        ASCII 'A') + 1

        nilaiBaru = nilaiSekarang + nilaiSebelumnya
        nilaiBaru = (nilaiBaru - 1) MODULO 26 + 1

        hurufBaru = Karakter dari (Nilai ASCII 'A' + nilaiBaru -
        1)

        TAMBAHKAN hurufBaru KE DALAM hasil
    AKHIR JIKA
AKHIR UNTUK

TAMPILKAN "Pesan setelah dienkripsi: ", hasil
SELESAI

```

(Pseudocode Problem 2)

Penjelasan Pseudocode Problem 2: Alien-In-The-Middle

1. Gambaran Umum

Pseudocode ini adalah **algoritma enkripsi sederhana** yang mengubah setiap huruf dalam pesan berdasarkan **huruf itu sendiri ditambah huruf sebelumnya**. Jadi ini bukan sandi Caesar biasa (yang menggeser semua huruf dengan jumlah tetap), melainkan pergeseran yang berubah-ubah sesuai isi pesan.

Algoritma ini mengasumsikan input hanya berisi **huruf kapital A-Z**.

2. Penjelasan Baris per Baris

```
Deklarasi variabel  
DEKLARASI pesan SEBAGAI String  
DEKLARASI hasil SEBAGAI String = ""
```

- pesan menampung teks asli yang dimasukkan pengguna.
- hasil menampung teks terenkripsi, dan awalnya kosong.

Input

```
TAMPILKAN "Masukkan pesan rahasia: "  
BACA pesan
```

Program meminta pengguna mengetik pesan, lalu menyimpannya ke pesan.

Perulangan

```
UNTUK i DARI 0 SAMPAI panjang(pesan) - 1 LAKUKAN
```

Setiap karakter diproses satu per satu, dari indeks 0 sampai karakter terakhir.

Kasus 1: karakter pertama ($i == 0$)

```
JIKA i == 0 MAKA  
    TAMBAHKAN pesan[i] KE DALAM hasil
```

Karakter pertama **tidak diubah**, karena belum ada karakter sebelumnya untuk dijumlahkan. Huruf ini langsung disalin ke hasil.

Kasus 2: karakter ke-2 dan seterusnya

a. Ubah huruf menjadi angka 1-26

```
nilaiSekarang = (ASCII pesan[i]) - (ASCII 'A') + 1  
nilaiSebelumnya = (ASCII pesan[i-1]) - (ASCII 'A') + 1
```

Kode ASCII 'A' adalah 65. Dengan mengurangi 65 lalu menambah 1, huruf dipetakan menjadi: A=1, B=2, C=3, ..., Z=26.

nilaiSekarang adalah nilai huruf saat ini, dan nilaiSebelumnya adalah nilai huruf **asli** tepat sebelumnya, yaitu huruf dari pesan, bukan dari hasil.

b. Jumlahkan kedua nilai

```
nilaiBaru = nilaiSekarang + nilaiSebelumnya
```

Hasilnya berkisar antara 2 sampai 52, sehingga bisa melebihi 26.

c. Bungkus kembali ke rentang 1-26

```
nilaiBaru = (nilaiBaru - 1) MODULO 26 + 1
```

Rumus ini menjaga hasil tetap di rentang 1-26 (huruf A-Z). Jika hasilnya melewati Z, hitungan kembali ke A. Bentuk $(x-1) \text{ MOD } 26 + 1$ dipakai karena nilai huruf dimulai dari 1, bukan 0.

d. Ubah angka kembali menjadi huruf

```
hurufBaru = Karakter dari (ASCII 'A' + nilaiBaru - 1)
TAMBAHKAN hurufBaru KE DALAM hasil
```

Ini kebalikan dari langkah (a). Angka 1 menjadi 'A', angka 2 menjadi 'B', dan seterusnya. Huruf hasilnya ditambahkan ke hasil.

Output

TAMPILKAN "Pesan setelah dienkripsi: ", hasil

3. Contoh Simulasi

Input: **HELLO**

i	Huruf	Nilai huruf	Nilai sebelumnya	Jumlah	Setelah modulo	Hasil
0	H	8	-	-	-	H (tidak diubah)
1	E	5	8 (H)	13	13	M
2	L	12	5 (E)	17	17	Q
3	L	12	12 (L)	24	24	X
4	O	15	12 (L)	27	$(27-1) \text{ mod } 26 + 1 = 1$	A

Output: **HMQXA**

Pada baris terakhir, jumlah 27 melewati 26 sehingga kembali ke awal alfabet dan menjadi 'A'.

4. Cara Dekripsi

Karena karakter pertama tidak berubah, pesan bisa dikembalikan secara berurutan:

- $P[0] = C[0]$
- $P[i] = (C[i] - P[i-1])$ dengan modulo 26, di mana P adalah plaintext dan C adalah ciphertext.

Setiap huruf yang berhasil didekripsi dipakai untuk mendekripsi huruf berikutnya.